

Chapter 3. Mathematical Considerations

Most of this book is focused on implementation and on practical considerations necessary to get recommendation systems working. In this chapter, you'll find the most abstract and theoretical concepts of the book. The purpose of this chapter is to cover a few of the essential ideas that undergird the field. It's important to understand these ideas as they lead to pathological behavior in recommendation systems and motivate many architectural decisions.

We'll start by discussing the shape of data you often see in recommendation systems, and why that shape can require careful thought. Next we'll talk about the underlying mathematical idea, similarity, that drives most modern recommendation systems. We'll briefly cover a different way of thinking about what a recommender does, for those with a more statistical inclination. Finally, we'll use analogies to NLP to formulate the popular approach.

Zipf's Laws in RecSys and the Matthew Effect

In a great many applications of ML, a caveat is given early: the distribution of observations of unique items from a large corpus is modeled by *Zipf's law*—the frequency of occurrence drops exponentially. In recommendation systems, the *Matthew effect* appears in the popular item's click rates or the popular user's feedback rates. For example, popular items have dramatically larger click counts than average, and more-engaged users give far more ratings than average.

The Matthew effect—or *popularity bias*—states that the most popular items continue to attract the most attention and widen the gap with other items.

Take, for example, the [MovieLens dataset](#), an extremely popular dataset for benchmarking recommendation systems. [Jenny Sheng](#) observes the behavior shown in [Figure 3-1](#) for a number of movie ratings:

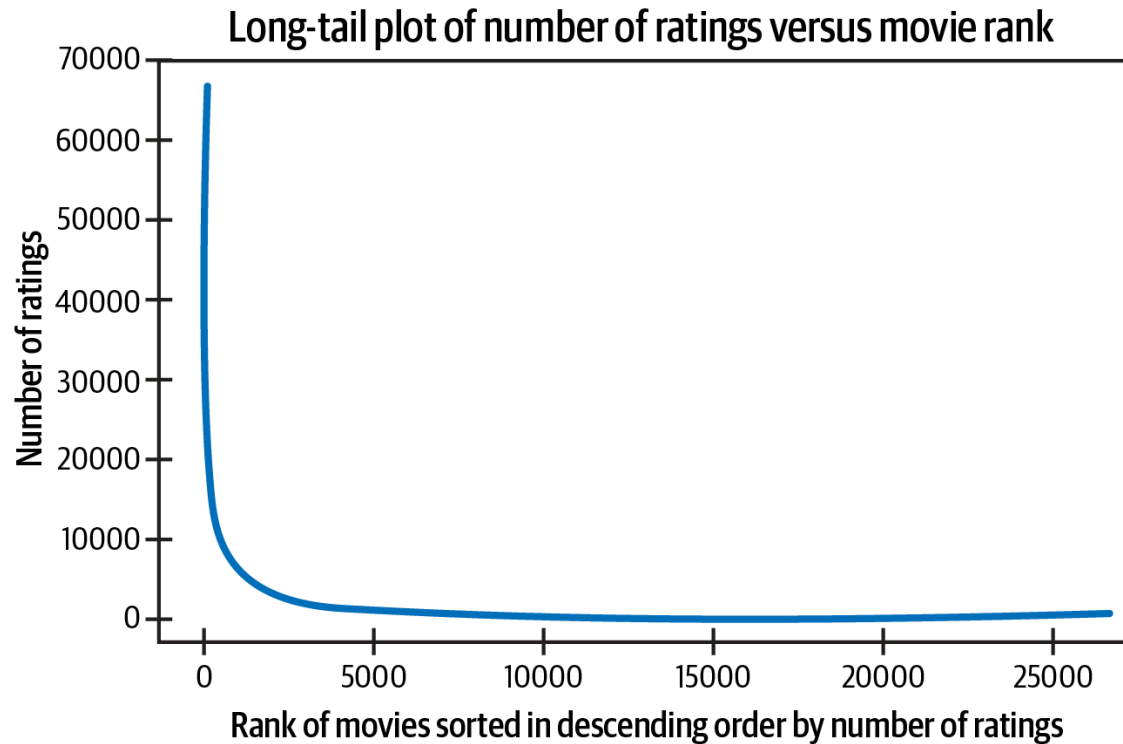


Figure 3-1. Zipfian distribution of movie-rank ratings

At first glance, the rapid decline in ratings is obvious and stark, but is it a problem? Let's assume our recommender will be built as a user-based collaborative filtering (CF) model—as alluded to in [Chapter 2](#). Then how might these distributions affect the recommender?

We will consider the distributional ramifications of this phenomenon. Let the probability mass function be described by the simple Zipf's law:

$$f(k, M) = \frac{1/k}{\sum_{n=1}^M (1/n)}$$

For M number of tokens in the corpus (in our example, the number of movies), k is the rank of a token when sorted by number of occurrences.

Let's consider users A and B , with $N_A = |\mathcal{J}_A|$ and $N_B = |\mathcal{J}_B|$ ratings, respectively. Observe that the probability of V_i , the i th most popular video, appearing in $\mathcal{J}_X$ for a user X is given by the following:

$$P(i) = \frac{f(i, M)}{\sum_{j=1}^M f(j, M)} = \frac{1/i}{\sum_{j=1}^M 1/j}$$

Thus the joint probability of an item appearing in two user's ratings is shown here:

$$P(i^2) = \left(\frac{1/i}{\sum_{j=1}^M 1/j} \right)^2$$

In words, the probability of two users sharing an item in their rating sets drops off with the square of its popularity rank.

This becomes important when we also consider that our, yet unstated, definition of user-based CF is based on similarity in users' ratings sets. This similarity is *the number of jointly rated items by two users, divided by the total number of items rated by either*.

Taking this definition, we can, for example, compute the similarity score for one shared item among A and B :

$$\sum_{i=1}^M \frac{P(i^2)}{\|\mathcal{J}_A \cup \mathcal{J}_B\|}$$

The average similarity score of two users is then generalized as follows via repeated application of the preceding equation:

$$\sum_{t=1}^{\min(N_A, N_B)} \prod_{i_k=i_{k-1}+1}^{t-1} \sum_{i=1}^M \frac{P(i_k^2)}{\frac{\|\mathcal{J}_A \cup \mathcal{J}_B\|}{t}} \Bigg)$$

via repeated application of the preceding observation.

These combinatorial formulas not only indicate the relevance of the Zipfian in our algorithms, but we also see an almost direct effect on the output of scores. Consider the experiment in [“Quantitative Analysis of Matthew Effect and Sparsity Problem of Recommender Systems”](#) by Hao Wang et al. on the [Last.fm dataset](#). Last.fm is a music-listening tracker en-

abling users to keep track of all the songs they listen to; for Last.fm users, the authors demonstrate average similarity scores for pairs of users, and they find that this Matthew effect persists into the similarity matrix (Figure 3-2).

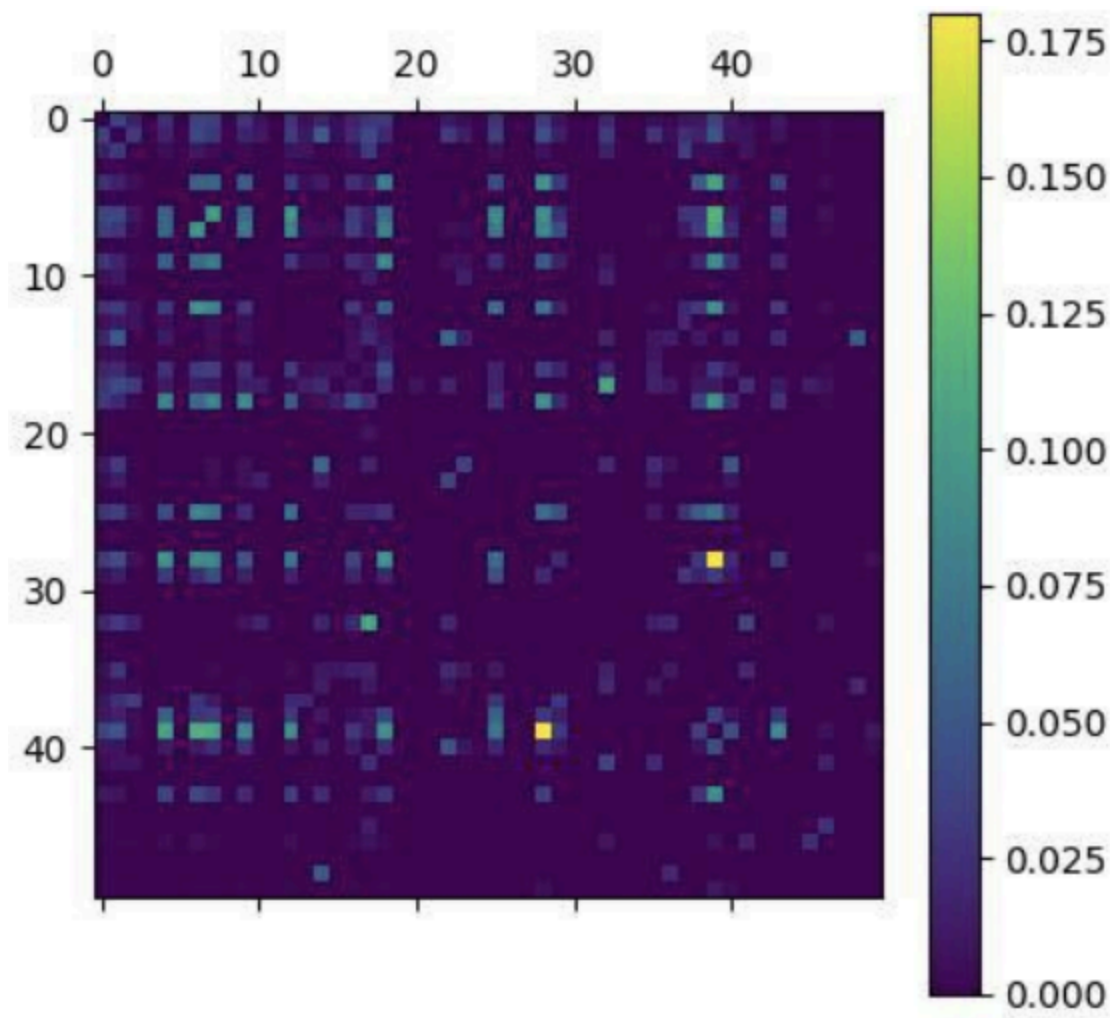


Figure 3-2. Matthew effect as seen on the Last.fm dataset

Observe the radical difference between “hot” cells and all the others. The bright cells are few among the mostly dark, suggesting a difficult combination of some extremely popular items among the far more common frequency close to zero. While these results might seem scary, later we’ll consider diversity-aware loss functions that can mitigate the Matthew effect. A simpler way is to use downstream sampling methods, which we will discuss as part of our explore-exploit algorithms. Finally, the Matthew effect is only the first of two major impacts of this Zipfian; let’s turn our attention to the second.

Sparsity

We must now reckon with sparsity. As the ratings skew more and more toward the most popular items, the least popular items are starved for data and recommendations, which is called *data sparsity*. This connects to the linear-algebraic definition: mostly zeros or not populated elements in a vector. When you consider again our user-item matrix, less popular items constitute columns with few entries; these are sparse vectors.

Similarly, at scale we see that the Matthew effect pushes more and more of the total ratings into certain columns, and the matrix becomes sparse in the traditional mathematical sense. For this reason, sparsity is an extremely well-known challenge for recommendation systems.

As before, let's consider the implication on our CF algorithms from these sparse ratings. Again observe that the probability of V_i , the i th most popular item, appearing in $\mathcal{J}_X$ for a user X is given by the following:

$$P(i) = \frac{f(i, M)}{\sum_{j=1}^M f(j, M)} = \frac{1/i}{\sum_{j=1}^M 1/j}$$

Then

$$(M - 1) * P(i)$$

is the expected number of other users who click the i th most popular item, so summing over all, i yields the total number of other users who will share a rating with X :

$$\sum_{i=1}^M (M - 1) * P(i)$$

Again, as we pull back to the overall trends, we observe this sparsity sneaking into the actual computations for our CF algorithms, consider the trend of users of different ranks, and see how much their rankings are used to *collaborate* in other users' rankings ([Figure 3-3](#)).

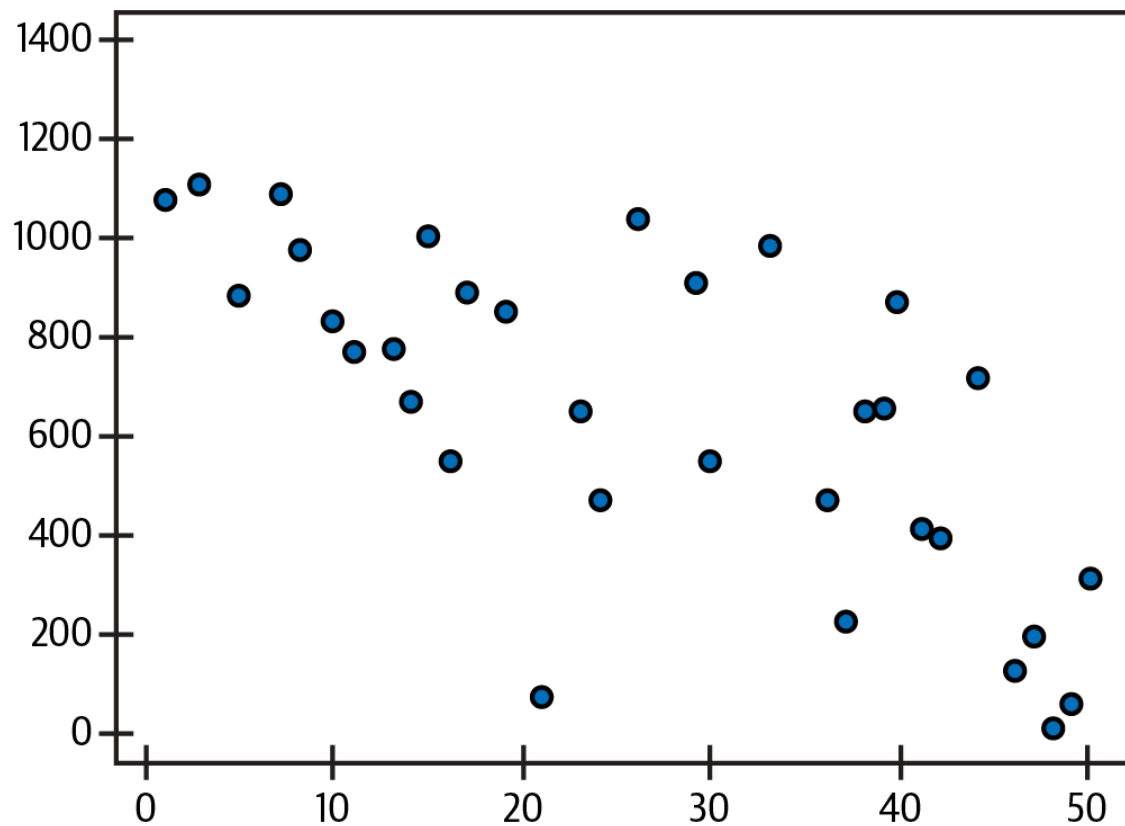


Figure 3-3. User similarity counts for the Last.fm dataset

We see that this is an important result to always be aware of: sparsity pushes emphasis onto the most popular users and has the risk of making your recommender myopic.

ITEM-BASED COLLABORATIVE FILTERING

While the equations are different, in this section, they apply similarly to item-based CF. Similarity in items exhibits the same inheritance of the Zipfian in their scores, and items consulted in the CF process drop off by rank.

User Similarity for Collaborative Filtering

In mathematics, it's common to hear discussion of *distances*. Even back to the Pythagorean theorem, we are taught to think of relationships between points as distances or dissimilarity. Indeed, this fundamental idea is canonized in mathematics as part of the definition of a metric:

$$d(a, c) \leq d(a, b) + d(b, c)$$

In ML, we often instead concern ourselves with the notion of similarity—an extremely related topic. In many cases, we can compute similarity or dissimilarity, as they are complements of each other; when $d : X \times X \rightarrow [0, 1] \subset \mathbb{R}$ is a *dissimilarity function*, then we often define the following:

$$\text{Sim}(a, b) := 1 - d(a, b)$$

This may seem like a needlessly precise statement, but in fact you'll see that a [variety of options are available for framing similarity](#).

Furthermore, sometimes we even formulate similarity measures where the associated distance measure does not establish a metric on the set of objects. These so-called pseudospaces can still be incredibly important, and we'll show where they come up in [Chapter 10](#).

In the literature, you'll find that papers commonly start by introducing a new similarity measure and then training a model you've seen before on that new measure. As you'll see, the way you choose to relate objects (users, items, features, etc.) can have a large effect on what your algorithms learn.

For now, let's laser in on some specific similarity measures. Consider a classic ML problem of clustering: we have a space (usually $\mathbb{R}^n$) in which our data is represented and are asked to partition our data into subcollections of the population and assign these collections names. Frequently, these collections are intended to capture a certain meaning, or at the very least be useful for summarizing the collection elements' features.

When you do that clustering, you frequently are considering points near to one another in that space. Further, if you're given a new observation and asked to assign it to a collection as an inference task, you normally compute the new observation's *nearest neighbors*. This could be the k -nearest neighbors or simply the nearest neighbor among cluster centers; either way, your task is to use the notion of similarity to associate—and thus classify. In CF, this same notion is used to relate a user for whom you wish to make recommendations to those you already have data from.

Nearest neighbors is a catchall term that arises from the simple geometric idea that, given some space (points defined by feature vectors) and a point in that space, you can find the other points closest to it. This has applications in all of ML, including classification, ranking/recommendation, and clustering. [“Approximate Nearest Neighbors”](#) provides more details.

So how can we define similarity for our users in CF? They’re not obviously in the same space, so our usual tools seem to be lacking.

Pearson Correlation

Our original CF formulation indicated that users with similar tastes collaborate to recommend items for one another. Let two users A and B have a set of co-rated items—simply the set of items with ratings from each—written as $\mathcal{R}_{A,B}$, and a rating of item x by user A written as $r_{A,x}$. Then the following is the sum of deviation from A ’s average rating over all of its co-rated items with B :

$$\sum_{x \in \mathcal{R}_{A,B}} (r_{A,x} - \bar{r}_A)$$

If we think of these ratings as a random variable and consider the analog for B , the correlation between the jointly distributed variables (the population covariance) is our *Pearson correlation*:

$$\text{USim}_{A,B} = \frac{\sum_{x \in \mathcal{R}_{A,B}} (r_{A,x} - \bar{r}_A) (r_{B,x} - \bar{r}_B)}{\sqrt{\sum_{x \in \mathcal{R}_{A,B}} (r_{A,x} - \bar{r}_A)^2} \sqrt{\sum_{x \in \mathcal{R}_{A,B}} (r_{B,x} - \bar{r}_B)^2}}$$

Keeping in mind a few details here is extremely important:

- This is the similarity of the jointly distributed variables describing the users’ ratings.
- We compute this via all co-rated items, so user similarity is defined via item-ratings.
- This is a pairwise similarity measure taking values in $[-1,1] \in \mathbb{R}$.

In [Part III](#), you will learn about additional definitions of *correlation* and *similarity* that are more well suited for handling ranking data and that accommodate implicit rankings in particular.

Ratings via Similarity

Now that we've introduced user similarity, let's use it! For a user A and item x , we can estimate the rating via similar users' ratings:

$$Aff_{A,i} = \bar{r}_A + \frac{\sum_{U \in \mathcal{N}(A)} \text{USim}_{A,U}^*(r_{U,i} - \bar{r}_A)}{\sum_{U \in \mathcal{N}(A)} \text{USim}_{A,U}}$$

This is the prediction for user A 's rating of item x , which takes A 's average adjusted rating of the similarity-weighted average ratings of all of A 's neighbors. In other words: A 's rating will probably be the average of people who have ratings like A 's rating, adjusted to how generous A is with ratings in general. We call this estimate the *user-item affinity score*.

But wait! What's $\mathcal{N}(A)$? It's the neighborhood of A , via our USim definition from the preceding section. The idea here is that we are aggregating ratings over the local region of users identified as similar to our target user by the previous USim metric. How many neighbors? How do you pick those neighbors? These will be the subject of later chapters; for now, assume they're k -nearest neighbors and assume that some hyperparameter tuning is used to determine a good value for k .

You might wonder, “Does this Pearson correlation yield a metric space under a transformation?” The answer is yes, but clearly defining the metric space is a bit more complicated than our simple definition. While the preceding equation can get us a distance, it’s not good enough to get us a metric space without a more novel transformation.

In particular, for $P(A, B)$, the previously defined correlation, $1 - P(A, B)$ yields a distance that satisfies all metric properties *except* the triangle inequality. There are several known ways to adjust this, though: $\sqrt{1 - P(A, B)^2}$ is the most common. For a survey, see [“Metric Distances Derived from Cosine Similarity and Pearson and Spearman Correlations”](#) by Stijn van Dongen and Anton J. Enright.

Explore-Exploit as a Recommendation System

So far we’ve presented two ideas, slightly in tension with each other:

- The MPIR, a simple, easy-to-understand recommender
- The Matthew effect in recommendation systems and its runaway behavior in distributions of ratings

By now, you likely realize that the MPIR will amplify the Matthew effect and that the Matthew effect will drive the MPIR to the trivial recommender in the limit. This is the classic difficulty of maximizing a loss function with no randomization: it quickly settles into a modal state.

This problem—and many others like it—encourages some modification to the algorithm to prevent this failure mode and continues to expose the algorithm and users to other options. The basic strategy for *explore-exploit* schemes, or *multiarmed bandits* as they’re called, is to take not only the outcome-maximizing recommendation but also a collection of alternative *variants*, and randomly determine which to use as a response.

Taking a step back: given a collection of variant recommendations, or *arms*, A , for which the outcome of each recommendation is y_t , we have a prior reward function $R(y_t)$. The bandit (called an *agent* in this literature) would like to maximize $R(y_t)$ but doesn't know the distribution of the outcomes $Y_{a \in A}$. The agent thus assumes some prior distributions for $Y_{a \in A}$ and then collects data to update those distributions; after sufficient observations, the agent can estimate the expected values of each distribution, $\mu_{a \in A} = \mathbb{E}(\mathcal{R}(Y_a))$.

If the agent was able to confidently estimate these reward values, the recommendation problem would be solved: at inference, the agent would simply estimate the reward values for all variants for the user and select the reward-optimizing *arm*. This is, of course, ridiculous in totality, but the basic idea is useful nonetheless: hold prior assumptions about what will be the greatest expected reward, and explore alternatives with some frequency to continue to update the distributions and refine your estimators.

Even when not explicitly using a multiarmed bandit, this insight is a powerful and useful framework for understanding the goal of a recommendation system. Utilizing the ideas of prior estimates for good recommendations and exploring other options to gain signal is a core idea that's recurring. Let's see one practicality of this approach.

ϵ -greedy

How often should you explore versus use your reward-optimizing arm? The first best algorithm is ϵ -greedy: for $\epsilon \in (0, 1)$, at each request the agent has the probability ϵ of choosing a random arm and the probability $1 - \epsilon$ of selecting the currently highest estimated reward arm.

Let's take the MPIR and slightly modify it to include some exploration:

```
from jax import random
key = random.PRNGKey(0)

def get_item_popularities() -> Optional[Dict[str, int]]:
```

```

...
    # Dict of pairs: (item-identifier, count item chosen)
    return item_choice_counts
return None

def get_most_popular_recs_ep_greedy(
    max_num_recs: int,
    epsilon: float
) -> Optional[List[str]]:
    assert epsilon < 1.0
    assert epsilon > 0

    items_popularity_dict = get_item_popularities()
    if items_popularity_dict:
        sorted_items = sorted(
            items_popularity_dict.items(),
            key=lambda item: item[1],
            reverse=True,
        )
        top_items = [i[0] for i in sorted_items]
        recommendations = []
        for i in range(max_num_recs): # we wish to return max_num_recs
            if random.uniform(key) > epsilon: # if greater than epsilon, exploit
                recommendations.append(top_items.pop(0))
            else: # otherwise, explore
                explore_choice = random.randint(1, len(top_items))
                recommendations.append(top_items.pop(explore_choice))
        return recommendations

return None

```

The only modification to our MPIR is that now we have two cases for each potential recommendation from our `max_num_recs`. If a random probability is less than our ϵ , we proceed as before and select the most popular; otherwise, we select a random recommendation.

We're interpreting maximization of reward as selecting the most-popular items. This is an important assumption, and as we move into more complicated recommenders, this will be the crucial assumption that we modify to get different algorithms and schemes.

Now let's summarize our recommender components again:

Collector

The collector here need not change; we still want to get the item popularities first.

Ranker

The ranker also does not change! We begin by ranking the possible recommendations by popularity.

Server

If the collector and ranker remain the same, clearly the server is what must be adapted for this new recommender. This is the case; instead of taking the top items to fill `max_num_recs`, we now utilize our ϵ to determine at each step if the next recommendation added to our list should be next in line from the ranker or a random selection. Otherwise, we adhere to the same API schema and return the same shape of data.

What Should ϵ Be?

In the preceding discussion, ϵ is a fixed number for the entire call, but what should the value be? This is actually an area of great study, and the general wisdom is to start with large ϵ (to encourage more exploration) and then reduce over time. Determining the rate at which you decrease it, the starting value, and so on, requires serious thought and research. Additionally, this value can be tied into your prediction loop and be part of the training process. See [“The Exploration-Exploitation Trade-Off: Intuitions and Strategies”](#) by Joseph Rocca for a deeper dive.

Other—often better—sampling techniques exist for optimization.

Importance sampling can utilize the ranking functions we build later to integrate the explore-exploit with what our data has to teach.

The NLP-RecSys Relationship

Let's utilize some intuition from a different area of ML, natural language processing. One of the fundamental models in NLP is *word2vec*: a sequence-based model for language understanding that uses the words that occur in sentences together.

For *skipgram-word2vec*, the model takes sentences and attempts to learn the implicit meaning of their words via their co-occurrence relationships with other words in those sentences. Each pair of co-occurring words constitutes a sample that is one-hot encoded and sent into a vocabulary-sized layer of neurons, with a bottleneck layer and a vocabulary-sized output layer for probabilities that words will occur.

Via this network, we reduce the size of our representation to the bottleneck dimension and thus find a smaller dimensional representation of all our words than the original corpus-sized one-hot embedding. The thinking is that similarity of words can now be computed via vector similarity in this new representation space.

Why is this related to recommendation systems? Well, because if we take the ordered sequence of user-item interactions (e.g., the sequence of movies a user has rated), we can utilize the same idea from *word2vec* to find item similarity instead of word similarity. In this analogy, the user history is the *sentence*.

Previously, using our CF similarity, we decided that similar users can help inform what a good recommendation for a user should be. In this model, we are finding item-item similarity, so instead we assume that a user will like the items similar to those previously liked.

You may have noticed that natural language models treat words as sequences, and in fact, our user history is a sequence too! For now, hold onto this knowledge. Later, this will guide us to sequence-based methods for RecSys.

Vector Search

We have built a collection of vector representations of our items, and we claim that similarity in this space (often called a *latent space*, *representation space*, or *ambient space*) means similarity in *likability* to users.

To convert this similarity to a recommendation, consider a user A with a collection of previously liked items $\mathcal{R}_A$, and consider $\mathcal{A} = \{v_x | x \in \mathcal{R}_A\}$ the set of vectors associated to those items in this latent space. We are looking for a new item y that we think is good for A .

THE OLD CURSE

These latent spaces tend to be of high dimension, which Euclidean distance famously performs poorly in. As regions become sparse, the distance function performance decreases; local distances are meaningful, but global distances are not to be trusted. Instead, cosine distance shows better performance, but this is a topic of deep exploration. Additionally, instead of minimizing the distance, in practice it's better to maximize the similarity.

One simple way to use similarity to produce a recommendation is to take the closest item to the average of those that A likes:

$$\operatorname{argmax}_y \operatorname{USim}(v_y, \operatorname{avg}(\mathcal{A})) \mid y \in \operatorname{Items}\}$$

Here, $d(-, -)$ is a distance function in the latent space (usually cosine distance).

The argmax essentially treats all of A 's ratings equally and suggests something near those. In practice, this process is often fraught. First, you could weight the terms by rating:

$$\left. \operatorname{argmax}_y \operatorname{USim}\left(v_y, \frac{\sum_{v_x \in \mathcal{A}} r_x}{|\mathcal{R}_y|}\right) \mid y \in \text{Items} \right\}$$

This can potentially improve the representativeness of the user feedback in the recommendations. Alternatively, you might find that a user rates movies across a variety of genres and themes. Averaging here will definitely lead to worse results, so maybe you want to simply find recommendations similar to one movie the user liked, weighted by that rating:

$$\left. \operatorname{argmax}_y \frac{\operatorname{USim}(v_y, v_x)}{r_x} \mid y \in \text{Items}, v_x \in \mathcal{A} \right\}$$

Finally, you may even want to do this process several times for different items a user liked to get k recommendations:

$$\left. \min - k \operatorname{argmax}_y \frac{\operatorname{USim}(v_y, v_x)}{r_x} \mid y \in \text{Items} \right\} \mid v_x \in \mathcal{A}$$

Now we have k recommendations; each is similar to something that the user has liked and is weighted by how much they liked it. This approach utilized only an implicit geometry of the items formed by their co-occurrences.

Latent spaces and the geometric power that comes with them for recommendations will be a through line for the rest of the book. We will often formulate our loss functions via these geometries, and we'll exploit the geometric intuition to brainstorm where to expand our technique next.

Nearest-Neighbors Search

A reasonable question to ask is “How do I get these vectors that minimize this distance?” In all the preceding schemes, we are computing many distances and then finding minimums. In general, the problem of nearest neighbors is an extremely important and well-studied question.

While finding the exact nearest neighbors can sometimes be slow, a lot of great progress has been made on approximate nearest neighbors (ANN) searches. These algorithms not only return very close to the actual nearest neighbors, but they also perform orders of complexity faster. In general, when you see us (or other publications) computing an **argmin** (the

argument that minimized the function) over some distances, there's a good chance ANN is what's used in practice.

Summary

Recommendation systems in the preceding chapter discussed data distribution principles such as Zipf's law and the Matthew Effect. These principles lead to challenges, such as skewed user similarity scores and data sparsity. In the world of ML, while the traditional math focuses on distance, the emphasis is on the concept of similarity. Different measures of similarity can drastically alter algorithm learning outcomes, with clustering being a primary application.

In the realm of recommendations, items are often represented in high-dimensional latent spaces. Similarity in these spaces hints at user preferences. Methods include recommending items close to a user's average liked items, and this may be improved by adding a weighting by user-rating. However, individual preferences necessitate diverse recommendations. Latent spaces continue to be influential, driving recommendation techniques.

Locating these vectors effectively requires the nearest-neighbors search. Though exact methods are resource-intensive, approximate nearest-neighbors offer a fast, precise solution, providing the foundation for the recommendation systems discussed in the current chapter.